

Patrón: Sección Crítica

Enfoque I — Fiabilidad de Hilos (Thread-Safety)

Una **Sección Crítica** es un fragmento de código que debe ser ejecutado por un único hilo a la vez para producir los resultados esperados. Cuando más de un hilo intenta ejecutar ese fragmento, pueden ocurrir resultados impredecibles.

Diferencias clave con el patrón Monitor

- La Sección Crítica es una forma más estricta del Monitor.
- Un Monitor bloquea un único objeto; una Sección Crítica bloquea toda la clase de objetos.
- En Java, se implementa combinando las palabras clave `static` y `synchronized`.
- Con el Monitor, dos hilos pueden ejecutar código sincronizado en objetos distintos. Con la Sección Crítica, ningún hilo puede ejecutar el código bloqueado en ningún objeto de la clase.

Problema: FileLogger en entornos multihilo

La clase `FileLogger` implementa el patrón Singleton. Su método `getFileLogger()` verifica si la variable `logger` es nula y, de ser así, crea una nueva instancia. En un entorno de un solo hilo esto funciona correctamente; sin embargo, en un entorno multihilo dos hilos pueden ejecutar `getFileLogger()` simultáneamente, provocando que el constructor de `FileLogger` se invoque dos veces.

Implementación sin protección (problemática)

```
public class FileLogger implements Logger {
    private static FileLogger logger;

    private FileLogger() { }

    public static FileLogger getFileLogger() {
        if (logger == null) {
            logger = new FileLogger();    // ¡Riesgo de doble inicialización!
        }
        return logger;
    }

    public synchronized void log(String msg) {
        FileUtil futil = new FileUtil();
        futil.writeToFile("log.txt", msg, true, true);
    }
}
```

Solución: convertir `getFileLogger()` en Sección Crítica

Declarar el método de clase `getFileLogger()` como `synchronized` lo convierte en una Sección Crítica. Esto garantiza que ningún par de hilos ejecute el método al mismo tiempo, eliminando la posibilidad de que el constructor se invoque más de una vez.

```
public class FileLogger implements Logger {
    private static FileLogger logger;

    private FileLogger() { }

    // Sección Crítica: static + synchronized
    public static synchronized FileLogger getFileLogger() {
        if (logger == null) {
            logger = new FileLogger();
        }
        return logger;
    }

    public synchronized void log(String msg) {
        FileUtil futil = new FileUtil();
        futil.writeToFile("log.txt", msg, true, true);
    }
}
```

¿Por qué funciona?

Al usar `static synchronized`, el bloqueo se aplica al objeto `Class` (nivel de clase), no a una instancia particular. Esto asegura que, aunque dos hilos operen sobre instancias distintas, solo uno a la vez pueda ejecutar `getFileLogger()`.

Consideración de rendimiento

Los métodos sincronizados pueden ser hasta **100 veces más lentos** que sus equivalentes no sincronizados. Si el rendimiento es crítico, considere el Enfoque II (inicialización temprana), que elimina la necesidad de sincronización.

Preguntas de práctica

1. Diseñe una clase de conexión a base de datos como un singleton seguro para hilos.
2. Diseñe una clase de cola de impresión (*spooler*) como un singleton seguro para hilos.